JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 08 OF 12 - STUDY STEP FW-08

Redis for Java Backend Interviews

Data Structures, Caching, Expiration, Coordination, and Failure-Aware Design

BY

Vinay Reddy Kalluri

A guide to choosing Redis structures and designing cache, expiration, rate-limit, and coordination behavior safely.

REDIS | CACHING | TTL | RATE LIMITING | CONSISTENCY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `FW 07 — MongoDB`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Teach Redis as a distinct system with explicit consistency and failure boundaries, not merely a fast map.

Readiness map

Decision	Evidence
START HERE	Latency motivates a cache or in-memory data structure; TTL and invalidation affect correctness; A rate limit, lock, or hot key must survive concurrency.
PREPARE FIRST	Database fundamentals; FW-04 Spring Boot recommended.
MOVE ON WHEN	Choose Redis structures and commands from access and atomicity requirements; Design TTL, eviction, caching, rate limits, and locks with failure safety; Operate replication, Sentinel, Cluster, and Java or Spring clients deliberately.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

FW 07 - MongoDB	**YOU ARE HERE** **FW 08 - Redis**	FW 09 - Persistence, SQL, and Caching

A note from Vinay

Redis is fast, but a cache is a second stateful system. Write the freshness, expiry, failure, and ownership rules before choosing a key or data type.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Redis for Java Backend Interviews

Learning Path: Bytes and Data Structures Before Caching

Redis is a networked data-structure server. Keys and most values are byte sequences; commands apply atomic transformations to structures such as strings, hashes, sets, sorted sets, lists, and streams. "Fast cache" describes only one use.

> **From Vinay:** Before saying "put it in Redis," write the key, value structure, command, TTL, source of truth, and failure response. A cache that is fast but returns forbidden or stale data is still incorrect.

Dependency order

TEXT

```
RESP command and keyspace
  -> data-structure semantics
  -> TTL, memory, eviction, persistence
  -> cache consistency and stampedes
  -> atomic commands, transactions, scripts/functions
  -> rate limits and coordination
  -> replication, Sentinel/Cluster, failover
  -> Java/Lettuce/Spring Data boundary
```

One running keyspace

TEXT

```
order:{42}:summary        STRING/JSON, TTL 5m
order:{42}:items          HASH, bounded fields
customer:{7}:recent       ZSET score=epochMillis member=orderId
rate:{tenant-9}:checkout  ZSET of request timestamps
stream:order-events       STREAM with consumer groups
lock:order:{42}           STRING random owner token + short TTL
```

The braces are Redis Cluster hash tags: keys sharing the same tag map to one slot. Use them only when an atomic multi-key operation truly needs co-location; a popular tag can create a hot slot.

The command path

```text
Java object
 -> serializer -> bytes
 -> client pool/native connection/event loop
 -> RESP command over TCP/TLS
 -> Redis parses + executes command on owning node
 -> memory/persistence/replication work
 -> RESP response
 -> decoder -> Java result
```

Latency can occur in client queueing, DNS/connect/TLS, network, command execution, slow scripts, memory pressure, persistence forks/rewrite, replication, cluster redirection, or decoding. "Redis took 2 ms" must identify which interval was measured.

Command atomicity is not workflow atomicity

`INCR`, `SET key value NX PX 5000`, and a Lua/function invocation execute atomically on the relevant Redis server context. A Java sequence `GET`, calculate, `SET` is not atomic because other clients can run between commands. A Redis command cannot roll back a MySQL commit, Kafka publish, or email.

The SDE-2 answer frame

1. What is the source of truth?
2. Which Redis structure and exact commands fit the operation?
3. What are the TTL, invalidation, eviction, and stale-data contracts?
4. Which concurrency race exists between commands?
5. What happens on timeout, failover, duplicate retry, or partial outage?
6. How do keys distribute across nodes and tenants?
7. Which metrics and repair path prove correctness?

First failure matrix

Failure	Risk	Safe posture
cache miss	extra source load	bounded fallback and stampede control
command timeout	write outcome unknown	idempotent command/token/reconciliation
eviction before TTL	unexpected miss	treat cache as disposable unless designed otherwise
replica/failover lag	acknowledged data may be lost/stale under policy	choose durability/consistency per use
hot key	one node/slot saturates	local cache, shard key, aggregate, or redesign

Practice

- **Foundation:** For three use cases, name the structure and command-not just "Redis."
- **Interview Core:** Define cache source of truth, TTL, invalidation, and outage behavior.
- **SDE-2 Follow-up:** Explain why a successful Redis lock does not make a MySQL-plus-HTTP workflow atomic.

SDE-2 CORE CHAPTER

Chapter 2 - RESP, Command Execution, Data Types, and Key Design

RESP framing

Redis clients encode commands using the Redis Serialization Protocol. A conceptual RESP2 request:

TEXT

```
*3\r\n
$3\r\nSET\r\n
$7\r\norder:1\r\n
$4\r\nPAID\r\n
```

The length prefixes make binary-safe values possible. Production clients pipeline, multiplex, reconnect, follow cluster redirects, authenticate, negotiate protocol version, and decode push/aggregate types. Do not hand-roll RESP in an application; the lab implements a small encoder only to expose the wire boundary.

Execution model without slogans

Redis traditionally executes commands serially on a server execution path, which makes one command atomic relative to other commands. Modern versions can use additional threads for I/O/background work, and clusters execute on multiple nodes. Therefore "Redis is single-threaded" is incomplete and "all operations are globally serialized" is false.

A long Lua script, large key deletion, expensive sorted-set range, or huge response can delay unrelated commands on that node. Complexity and payload size still matter.

Strings

Strings hold bytes and support `GET`, `SET`, `MGET`, `INCRBY`, bit operations, and conditional/TTL options.

TEXT

```
SET order:42:status PAID NX EX 300
INCRBY inventory:book-1 -1
```

`INCRBY` is atomic but does not enforce "never below zero" alone. Use a script/function that checks and decrements atomically, or model a reservation command.

Hashes

Hashes map fields to values under one key:

```text
HSET order:42 status PAID totalCents 5000 version 4
HINCRBY order:42 version 1
```

They suit bounded objects with partial field updates. TTL applies to the key unless using version-specific field-expiration features; label the target version before relying on newer commands.

Lists, sets, and sorted sets

- **List:** ordered sequence, push/pop and blocking operations. Good for simple queues, but reliability/replay need explicit design.
- **Set:** unique unordered members, membership and set algebra.
- **Sorted set:** unique members ordered by floating-point score; ranks, leaderboards, delayed timestamps, sliding windows.

Sorted-set scores are IEEE-754 doubles. Very large integer timestamps/counters can lose exact integer precision; keep the safe range or encode tie-breakers in members.

Streams

Streams are append-only entries with IDs, range reads, and consumer groups. Consumer groups track pending entries and acknowledgements, but processing is not automatically exactly once. Consumers can crash after the side effect and before `XACK`; the message will be delivered/claimed again. Make the effect idempotent and observe pending/idle entries.

Kafka offers a partitioned durable log with different scaling/retention/consumer semantics; Redis Streams are not a drop-in synonym.

Other structures

Bitmaps/bitfields, HyperLogLog, geospatial indexes, probabilistic structures, JSON, time-series, and vector structures fit specialized questions. State accuracy, module/version, memory, and query requirements. Do not force them into basic caching because they sound advanced.

Key naming and tenancy

Use stable, bounded names with namespace, tenant, entity, and version when useful:

```text
prod:checkout:v2:{tenant-9}:rate
```

Never put secrets/PII in keys; keys appear in diagnostics and consume memory. Estimate total key/value overhead and TTL population. Avoid unbounded `KEYS pattern` in production; use indexed ownership, `SCAN` with caveats, or administrative tooling.

WATCH OUT | Structure edge cases

Trap	Effect	Repair
giant hash/list	node stalls/large transfer	bound and partition
blocking queue as durable workflow	loss/replay gaps	stream/broker with explicit delivery contract
floating score as arbitrary integer	precision/tie surprise	safe numeric range + deterministic member
multi-key command across slots	cluster error	hash tag or redesign
`KEYS` in hot path	scans keyspace/blocking risk	explicit index/`SCAN` for admin only

Practice

- **Foundation:** Choose structures for unique tags, leaderboard, recent orders, and object fields.
- **Interview Core:** Explain stream pending entries and duplicate processing.
- **SDE-2 Follow-up:** Redesign a 50 MB hot hash with per-field updates and uneven tenant load.

SDE-2 CORE CHAPTER

Chapter 3 - TTL, Expiration, Eviction, Memory, Persistence, and Durability

Expiration is a contract on a key

```text
SET session:abc payload EX 1800
EXPIRE session:abc 1800
TTL session:abc
```

Setting a value can remove/preserve TTL depending on command/options/version. Test the exact update path. A key may be removed lazily on access or actively by expiration cycles, so physical deletion is not an exact scheduled event.

Authorization must compare logical expiry in the value/request, not wait for background deletion.

TTL jitter

If one million cache keys receive the same 10-minute TTL, they can expire together and overload the source. Add bounded random/deterministic jitter:

```text
effective TTL = base TTL + uniformly distributed 0..jitter
```

The jitter window should reflect freshness tolerance and traffic. For tests, derive it deterministically from the key to avoid flaky timing.

Expiration versus eviction

- **Expiration:** key TTL elapsed.
- **Eviction:** Redis removes keys under configured memory pressure according to a policy.

`noeviction` rejects writes when memory limits are reached; LRU/LFU/random/TTL-oriented policies choose candidates from all or expiring keys depending on configuration. Algorithms may be approximated. A cache must tolerate early eviction. A correctness-critical lock/rate counter sharing an evicting cache can disappear early and weaken guarantees.

Separate workloads or reserve policy/capacity when semantics differ.

Memory budgeting

Budget more than serialized payload:

```text
key bytes + value bytes + object/allocator overhead
+ expiry metadata + data-structure/index nodes
+ replication/AOF buffers + fragmentation + fork/COW headroom
```

Many tiny keys can use more memory in overhead than payload. One huge key can block commands and concentrate network. Measure `MEMORY USAGE`, allocator fragmentation, key distribution, and client output buffers on representative data.

RDB and AOF

- **RDB snapshots:** compact point-in-time files; recovery can lose writes since the last snapshot.
- **AOF:** logs write commands and can fsync under configurable policy; rewrite compacts history.
- **Both:** can combine recovery properties and operational costs.

Persistence consumes CPU, disk, memory, and latency headroom. Fork/copy-on-write and AOF rewrite can amplify memory/disk pressure. Configuration determines durability; "Redis is in memory, so data is not durable" and "AOF means no loss" are both oversimplifications.

Replication is asynchronous in common deployments. Acknowledged writes can be lost during failover depending on acknowledgement/durability/topology. `WAIT` can ask for replica acknowledgement but does not turn Redis into a strongly consistent consensus system or guarantee fsync everywhere.

Restart and cold-cache behavior

A cache restart can create a thundering herd while the source is cold too. Prepare:

- request coalescing/single-flight;
- admission control and bounded concurrency;
- warming only valuable keys;
- stale-while-revalidate where allowed;
- source capacity for planned failure;
- precomputed snapshots only if freshness/security allow.

Durability decision table

Use case	Persistence expectation	Failure posture
disposable product cache	none required	source fallback
rate limiter	loss may temporarily loosen limit	decide fail-open/closed and isolate
job/stream state	recovery required	AOF/replication plus idempotent source/replay
distributed lock	TTL essential, value not durable history	fencing at protected resource
session	product/security decision	source/refresh strategy and logical expiry

Practice and solutions

- **Foundation:** Distinguish TTL expiry from memory eviction.
- **Interview Core:** Add jitter and single-flight to a synchronized-expiry cache.
- **SDE-2 Follow-up:** Choose RDB/AOF/replication for a stream consumer state. Start from RPO/RTO and replay/idempotency; verify actual fsync/failover with tests.

SDE-2 CORE CHAPTER

Chapter 4 - Cache Patterns, Consistency, Stampedes, Hot Keys, and Negative Caching

Cache-aside

TEXT

```
read:
  GET cache key
    hit -> decode + validate logical freshness
    miss -> read source -> SET value with TTL -> return

write:
  commit source change -> invalidate/update cache
```

The dangerous interval is between source commit and cache invalidation. If invalidation fails, stale data remains until TTL. A short TTL limits but does not eliminate inconsistency.

For database-first writes, delete cache after commit. Updating cache before database commit can publish rolled-back state. Updating after commit can still fail; use an outbox/change stream and idempotent invalidation when the freshness contract demands recovery.

Write-through and write-behind

- **Write-through:** application writes through a cache layer to source; centralizes policy but adds coupling/latency.
- **Write-behind:** cache acknowledges before asynchronous source persistence; improves latency but creates durability, ordering, replay, and conflict risk.
- **Refresh-ahead:** refresh before expiry for predictable hot keys; wastes work for cold keys.
- **Stale-while-revalidate:** serve bounded stale value while one caller refreshes.

Name who owns truth and what happens when either side is unavailable.

Stampede control

A naive miss causes every caller to query the database. Techniques:

1. per-key single-flight in one process;
2. distributed short lease for cross-instance refresh;
3. serve stale value while refresh runs;
4. TTL jitter;
5. bounded source concurrency/admission control;
6. prewarm known hot keys.

A refresh lease must expire, use an owner token, and not block serving safe stale data longer than necessary. If refresh fails, back off to avoid a failure storm.

Negative caching

Caching "not found" prevents repeated misses but can hide a newly created resource. Use a shorter TTL, version/namespace strategy, and invalidate on creation. Never negative-cache an authorization denial across principals unless the key includes every security dimension.

Cache key completeness

If response varies by tenant, locale, currency, permissions, experiment, or API version, the key must reflect it or the cached payload must be independently safe.

TEXT

```
bad:  profile:42
good: profile:v3:{tenant-9}:42:locale=en-US:permission-set=hash
```

Permission hashes add invalidation complexity; often do not cache security-filtered responses globally.

Hot keys

A key can saturate a node, network link, or client connection even if total cluster traffic looks moderate. Diagnose per-key/slot and payload size. Options:

- local near-cache for immutable/bounded-stale data;
- replicate/read from replicas only with an acceptable consistency contract;
- split aggregatable counters;
- remove giant payload/projection;
- add request coalescing;
- redesign access rather than random key suffixes that break consistency.

Serialization and versioning

JSON is debuggable but verbose; binary codecs are compact but require schema discipline. Defend against Java native deserialization risks. Include a schema version when necessary, tolerate compatible old fields, bound payload sizes, and distinguish "missing" from "decode failure." A decode failure should not trigger unlimited source retries.

Cache failure matrix

Event	Risk	Policy choice
Redis unavailable	source overload	circuit/bulkhead, fail-open/closed per data
stale authorization	data exposure	do not serve beyond security freshness
invalidation lost	old value until TTL	outbox/change event and versioned key
deserialize failure	retry storm	evict/quarantine + metric + bounded source read
hot-key expiry	synchronized miss	jitter + single-flight + stale window
old writer repopulates	stale-after-delete race	version token/key namespace or delete-after-write strategy

Interview exercise

Scenario: Read misses, thread A loads version 1. Source updates to version 2 and invalidates. A then writes stale version 1 into cache.

Solutions: Store/compare a version in the value, use versioned keys plus current-version pointer, prevent stale fill using a lease/token, or perform a second invalidation after the write. Choose based on complexity and staleness budget.

Practice

- **Foundation:** Draw cache-aside read and write timelines.
- **Interview Core:** Protect a hot product key during expiry and Redis outage.
- **SDE-2 Follow-up:** Cache permission-dependent data without cross-tenant/user leakage.

SDE-2 CORE CHAPTER

Chapter 5 - Atomicity, Transactions, Scripts/Functions, Rate Limits, and Locks

Atomic single commands first

Prefer one built-in command when it expresses the invariant. `SET key value NX PX ttl` atomically claims a lease; separate `SETNX` then `EXPIRE` can leave a permanent key if the client dies between them.

MULTI/EXEC

Redis transactions queue commands and execute them without interleaving at `EXEC`. They do not provide SQL-style rollback for a command that errors during execution. Syntax/queue-time errors and runtime type errors have different outcomes; inspect every result.

TEXT

```
MULTI
INCR counter
EXPIRE counter 60
EXEC
```

Another client cannot interleave commands during the execution block, but reads before `MULTI` are not protected.

Optimistic control with WATCH

TEXT

```
WATCH balance:42
GET balance:42
MULTI
SET balance:42 new-value
EXEC
```

If a watched key changes, `EXEC` aborts and the client retries from a fresh read. Keep retries bounded and avoid external effects inside the attempt. Under high contention, repeated aborts can amplify load.

Lua scripts and Redis Functions

A script/function can check and mutate atomically on the executing node:

LUA

```
local current = tonumber(redis.call('GET', KEYS[1]) or '0')
local requested = tonumber(ARGV[1])
if current < requested then
  return 0
end
redis.call('DECRBY', KEYS[1], requested)
return 1
```

Pass keys through `KEYS`, values through `ARGV`, keep execution short/deterministic, and make all cluster keys share a slot. Long scripts block command progress on that node. Version/deploy/test scripts as production code and handle the "write completed, reply lost" case.

Rate limiting

Fixed window

Increment a key and set TTL atomically. Simple, but permits bursts around window boundaries.

Sliding log

Use a sorted set of unique request IDs scored by time; atomically remove old entries, count, and insert if allowed. Accurate but memory grows with requests in the window.

Token bucket

Store tokens and last-refill time; script calculates refill and consumes atomically. Supports controlled bursts with bounded state. Use server time or carefully address client clock skew.

Every limiter needs scope/key, limit/window, clock, atomicity, TTL cleanup, fail-open/closed policy, and observability. A Redis outage cannot be answered universally: login abuse may fail closed; low-risk recommendation refresh may fail open.

Distributed locks and fencing

Acquire:

TEXT

```
SET lock:order:{42} random-owner-token NX PX 5000
```

Release only if value still matches, using a script. Never `DEL` blindly: the lease may expire and another owner may acquire before the old owner releases.

Even token-safe release does not stop a paused old owner from modifying a protected database after its lease expires:

```text
owner A lease=1 pauses
lease expires
owner B lease=2 writes
owner A resumes and writes stale result
```

A **fencing token** is a monotonically increasing lease number that the protected resource rejects when older than the last seen token. If the resource cannot enforce fencing/idempotent conditional writes, the Redis lock is only a best-effort coordination hint.

Edge matrix

Trap	Failure	Repair
`SETNX` then `EXPIRE`	permanent lock	atomic `SET NX PX`
`DEL` on release	deletes another owner's lease	compare token then delete
long script	node-wide latency	bounded atomic work
`MULTI` assumed rollback	partial runtime command errors	validate/types and inspect results
rate limiter client clock	skew/manipulation	server time or defined tolerance
retry after timeout	duplicate mutation	idempotent operation ID/result ledger

Practice and solution direction

- **Foundation:** Repair a two-command lock acquisition.
- **Interview Core:** Implement token bucket inputs and outputs as one script/function.
- **SDE-2 Follow-up:** Protect a database write with fencing: allocate token, include it in `UPDATE ... WHERE last_token < ?`, and reject stale owners.

SDE-2 CORE CHAPTER

Chapter 6 - Replication, Sentinel, Cluster, Failover, Operations, and Security

Replication and failover

Redis replication is commonly asynchronous. Replicas can serve reads under a stated staleness policy and provide promotion candidates. A primary can acknowledge a write that has not reached the promoted replica, so failover may lose it depending on topology and settings.

Sentinel monitors non-clustered primary/replica deployments, participates in failure detection/election, and tells clients the promoted primary. Clients must support Sentinel discovery and reconnect. Sentinel does not shard the keyspace.

Redis Cluster

Cluster maps keys to 16,384 hash slots distributed across primary nodes. Clients receive slot maps and follow `MOVED`/`ASK` redirections during steady routing/migration.

```text
key -> CRC16(hash-tag-or-full-key) mod 16384 -> slot -> node
```

Multi-key operations, scripts, and transactions generally require keys in one slot. Hash tags such as `{order-42}` force co-location. Do not put every key under `{global}`; that defeats distribution.

Cluster provides availability and partitioning with documented windows where writes can be lost or unavailable. It is not a consensus-backed linearizable database for arbitrary cross-slot invariants.

Read scaling

Replica reads trade freshness for capacity/latency. Cache-aside reads can often tolerate them; locks, rate limits, session revocation, and read-after-write may not. Confirm whether the client routes reads to replicas and how it reacts to lag/failover.

Failure timeline

```text
client sends INCR to primary
primary applies and replies
reply reaches client
primary fails before replica applies
stale replica promoted
counter increment disappears
```

If this counter enforces a billing or security invariant, Redis alone under that policy is the wrong source of truth. Make the durable system authoritative or choose a stronger design.

Operations

Monitor:

- command rate/latency and slow log;
- event-loop/client queueing, blocked clients, timeouts;
- memory used, fragmentation, allocator/fork headroom;
- evictions, expirations, hit/miss and stale/fallback outcomes;
- persistence fsync/rewrite/snapshot duration and errors;
- replication offset/lag, link state, failovers;
- cluster slot distribution, redirects, migrations, hot nodes/keys;
- client connections/output buffers;
- backup age and restore-test success.

Use `SCAN`-style administrative iteration cautiously; it can return duplicates/miss transient changes during mutation and still consumes work. Never run broad destructive key operations from an unchecked pattern.

Capacity and overload

Set `maxmemory` and eviction policy intentionally. Bound client connections, pipelines, response sizes, and command rate. When Redis slows, client retries can multiply traffic. Use deadlines, exponential backoff with jitter, circuit breakers, and source bulkheads.

Pipeline length is a memory/latency trade-off; a million queued commands are not a free batch. Cluster pipelines must group by owning node and handle partial errors/redirections.

Backup/restore and upgrades

If Redis holds recoverable state, back up RDB/AOF artifacts according to RPO/RTO and test restore plus application reconciliation. For pure cache, test cold-start/source protection instead. Upgrade clients and servers through compatibility testing for RESP, ACL, cluster, persistence, and command changes.

Security

- private network/TLS and authenticated ACL users;
- least command/key-prefix permissions;
- rotate credentials without reconnect storms;
- disable/restrict dangerous administration commands;
- never expose Redis directly to the public internet;
- avoid secrets/PII in keys, logs, and diagnostics;
- validate sizes/commands before scripts;
- isolate tenants/workloads when noisy-neighbor or policy needs it.

Incident matrix

Incident	Evidence	Correction
rising p99 but low CPU	slow command/large response/client queue	command and payload trace
one node overloaded	hot key/slot imbalance	key/slot distribution and redesign
failover loses counter	async replication window	source-of-truth/durability change
mass eviction	capacity/policy/payload growth	budget and separate critical state
reconnect storm	failover + eager retry	backoff/jitter and connection budgets

Practice

- **Interview Core:** Compare Sentinel and Cluster.
- **SDE-2 Follow-up:** Define an outage plan for cache, login rate limiter, and job stream; each needs a different fail-open/closed and recovery policy.

SDE-2 CORE CHAPTER

Chapter 7 - Java, Lettuce, Spring Data Redis, Serialization, Testing, and Diagnostics

Connection model

Lettuce uses Netty and supports thread-safe connection sharing for many nonblocking commands, while blocking/transactional/pub-sub usage often needs dedicated connection semantics. A pool is not automatically required for every ordinary asynchronous command; choose it for isolation/stateful operations with measured limits.

Never block an event-loop thread waiting for its own future. In reactive code, keep the entire path nonblocking and propagate cancellation/backpressure.

Native command first

JAVA

```java
String result = commands.set(
        key,
        ownerToken,
        SetArgs.Builder.nx().px(leaseMillis));
boolean acquired = "OK".equals(result);
```

For production lock release, execute a compare-and-delete script. Store tokens as opaque random bytes/strings and use a fencing mechanism at the protected resource when correctness requires it.

Spring Data Redis progression

1. Native client command establishes exact Redis semantics.
2. `RedisTemplate`/typed operations make serializers and operations explicit.
3. Spring Cache annotations fit simple cache contracts after key, TTL, invalidation, null, and stampede policy are known.

JAVA

```java
ValueOperations<String, OrderSummary> values = template.opsForValue();
OrderSummary cached = values.get(key);
if (cached != null) {
    return cached;
}
```

That code still lacks single-flight, logical freshness, null/decode distinction, source deadline, TTL jitter, and invalidation. An annotation cannot decide those product semantics.

Serialization

Configure key and value serializers explicitly. Avoid Java native serialization for untrusted or long-lived cache data. JSON needs stable type/version handling; generic polymorphic type metadata can create security and compatibility risks. Test:

- old/new fields and schema version;
- null versus missing key;
- corrupted/truncated payload;
- numeric/time precision;
- maximum size and compression limits;
- cross-service interoperability.

Pipelining and transactions in clients

Pipelining reduces round trips; replies still correspond to commands in order and each can fail. `MULTI/EXEC` has connection state, so keep the same dedicated connection and inspect results. In Cluster, split/group pipelines by slot/node or let a cluster-aware client do so under documented behavior.

Timeouts and unknown outcomes

Separate connect, command, pool, and total request deadlines. If a mutating command times out after send, the server may have applied it. Use idempotency tokens/conditional commands and do not retry non-idempotent scripts blindly.

Testing layers

- deterministic Java tests for TTL, jitter, limiter, fencing, and RESP encoding;
- client command-shape tests;
- integration against exact Redis release/topology;
- failover/cluster movement/persistence restore tests;
- load tests for hot keys, scripts, payloads, and reconnect storms.

Embedded substitutes rarely reproduce eviction, fork/AOF pressure, replication loss, Cluster redirects, or timing. The included lab intentionally validates logic without claiming server behavior.

Diagnostic sequence

```text
request -> cache decision (hit/miss/stale/bypass)
        -> connection/queue wait
        -> command + key namespace hash + payload bytes
        -> node/slot + server duration
        -> decode/fallback/source duration
```

Track hits that were unusable due to decode/version/security separately from normal misses.

WATCH OUT | Edge cases

Mistake	Effect
default serializer changes	old data unreadable
new client per command	connection/TLS churn
huge pipeline	memory and timeout spike
generic retry on timeout	duplicate increment/script
cache annotation on self-invocation	proxy advice may not run
blocking call on reactive loop	stalled throughput

Practice and solutions

- **Foundation:** Define serializers and a key version for an order summary.
- **Interview Core:** Instrument hit, stale, decode failure, fallback, and source overload separately.
- **SDE-2 Follow-up:** Test a failover during a non-idempotent increment; show why the durable source or idempotent operation ledger must decide the final value.

SDE-2 CORE CHAPTER

Chapter 8 - Redis Live Interviews, Rapid Q&A, Practice, and Sources

Live interview 1: cache-aside race

Interviewer: "A stale value reappears after invalidation."

Candidate: "A reader missed, loaded v1, a writer committed v2 and deleted the cache, then the reader filled v1. I would use a versioned value/key, refresh lease plus version check, or second invalidation. TTL only bounds the error; it does not prevent it."

Live interview 2: stampede

Interviewer: "Ten thousand requests miss one key."

Candidate: "Coalesce refresh per key, serve bounded stale data if safe, jitter TTLs, cap source concurrency, and back off failed refreshes. A distributed lease coordinates instances but uses owner tokens/expiry and must not make the cache a permanent dependency."

Live interview 3: rate limiter

Interviewer: "Design API rate limiting."

Candidate: "Define scope, limit/window, burst, accuracy, clock, and outage policy. Fixed window is cheap but boundary-bursty; sliding log is exact but memory-heavy; token bucket gives controlled bursts. One script/function makes trim/refill/check/update atomic, sets cleanup TTL, and returns remaining/reset metadata."

Live interview 4: distributed lock

Interviewer: "Is `SET NX PX` enough?"

Candidate: "It provides a lease claim. Release must compare owner token. A paused owner can resume after expiry, so the protected database needs a monotonically increasing fencing token or its own version/conditional write. I keep lease work bounded and treat timeout outcome carefully."

Live interview 5: Redis Cluster cross-slot error

Interviewer: "Our script fails only in production cluster."

Candidate: "Its keys map to different slots. Use a deliberate shared hash tag when atomic co-location is required, or redesign into independent operations with reconciliation. I would ensure the tag does not concentrate all tenants into one slot."

Live interview 6: failover loses data

Interviewer: "An acknowledged counter decreased after promotion."

Candidate: "Common Redis replication is asynchronous; the promoted replica may not have applied the write. I would inspect acknowledgement/persistence/topology and decide whether Redis is allowed to be authoritative. For billing/security truth, store an idempotent durable ledger and use Redis as acceleration."

Live interview 7: slow Redis

Interviewer: "CPU is low but p99 is high."

Candidate: "Break down client queue/connection, network, command execution, response size, and decoding. Inspect slow commands, big/hot keys, blocked clients, persistence fork/rewrite, output buffers, cluster redirects, and retry amplification. Low CPU does not rule out event-loop stalls or network/large payloads."

Rapid answered questions

1. **Is Redis just a cache?** No; it is a data-structure server with caching, coordination, streams, and more.
2. **Are all workflows atomic?** One command/script block can be atomic on its node; multi-system workflows are not.
3. **TTL versus eviction?** Expiration follows time; eviction follows memory policy and may occur earlier.
4. **Is expiry exact?** Logical expiry is exact by time check; physical deletion can be delayed.
5. **Pipeline versus transaction?** Pipeline batches network traffic; transaction prevents command interleaving during `EXEC` but has no SQL-style rollback.
6. **What does `WATCH` do?** Aborts `EXEC` if watched state changed, enabling optimistic retry.
7. **Why scripts must be short?** They block other command execution on that node while running.
8. `SETNX` **plus** `EXPIRE`**?** Race leaves permanent lock; use atomic `SET NX PX`.
9. **Why random lock token?** Only the owner should release its still-current lease.
10. **Why fencing?** It lets the protected resource reject a stale paused owner.
11. **Sentinel versus Cluster?** Sentinel discovers/fails over one unsharded primary group; Cluster partitions slots and fails over shard primaries.
12. **Can replica reads be stale?** Yes; use only when the data contract permits it.
13. **Is replication backup?** No; it copies mistakes and lacks historical recovery by itself.
14. **Why hot key?** One key maps to one owning node/slot and can dominate compute/network.
15. **Do Streams give exactly once?** No; crash between effect and ack produces redelivery; make effects idempotent.
16. **Should cache failure fail open?** It depends on security/correctness. Decide per use case, not globally.
17. **Why no `KEYS` in hot path?** It scans the keyspace and can delay the node.
18. **Why serializer versioning?** Cached bytes can outlive deployments and be read by mixed versions.

Cumulative assessment

Design Redis support for product cache, checkout rate limit, leaderboards, and a cross-instance refresh lease. Include keys/structures/commands, TTL+jitter, source truth, atomic scripts, cluster slots, failover outcomes, memory estimate, security ACLs, dashboards, and restore/cold-start tests.

Strong solution: isolates disposable cache from correctness-sensitive state, prevents stampedes, uses idempotency/fencing, handles unknown outcomes, distributes keys, and defines per-use-case outage behavior.

Authoritative references

- Redis documentation: data types, commands, pipelining, transactions, scripting/functions, expiration, eviction, persistence, replication, Sentinel, Cluster, Streams, ACLs, and observability.
- Lettuce reference: connection, async/reactive, cluster, Sentinel, topology refresh, codecs, and timeouts.
- Spring Data Redis reference: templates, serializers, cache, transactions, scripting, streams, repositories, and observability.

The included lab proves deterministic Java logic and RESP framing, not live-server eviction, replication, failover, persistence, or Cluster behavior.

SDE-2 CORE CHAPTER

Chapter 9 - Java 21 Redis Interview Reasoning Companion

Executable models for structure selection, TTL jitter, logical freshness, Cluster hash tags, lease ownership, and binary-safe encoding.

JAVA (continued 1/3)

```java
import java.nio.charset.StandardCharsets;
import java.util.Objects;

/** Dependency-free executable reasoning models for the Redis volume. */
public final class RedisInterviewCompanion {
    private RedisInterviewCompanion() {}

    enum Structure { STRING, HASH, SET, SORTED_SET, STREAM }

    record Requirement(boolean uniqueMembers, boolean scoreOrdered,
                       boolean fieldUpdates, boolean replayableLog) {}

    static Structure chooseStructure(Requirement requirement) {
        Objects.requireNonNull(requirement, "requirement");
        if (requirement.replayableLog()) {
            return Structure.STREAM;
        }
        if (requirement.scoreOrdered()) {
            return Structure.SORTED_SET;
        }
        if (requirement.uniqueMembers()) {
            return Structure.SET;
        }
        return requirement.fieldUpdates() ? Structure.HASH : Structure.STRING;
    }

    static long deterministicTtlSeconds(String key, long base, long jitter) {
        Objects.requireNonNull(key, "key");
        if (base <= 0 || jitter < 0) {
            throw new IllegalArgumentException("invalid TTL");
        }
```

JAVA (continued 2/3)

```
        long offset = jitter == 0 ? 0 : Math.floorMod(key.hashCode(), jitter + 1);
        return Math.addExact(base, offset);
    }

    record CacheValue(String payload, long sourceVersion, long logicalExpiresAtMillis) {
        CacheValue {
            Objects.requireNonNull(payload, "payload");
        }

        boolean freshAt(long nowMillis) {
            return nowMillis < logicalExpiresAtMillis;
        }
    }

    static String clusterHashTag(String key) {
        Objects.requireNonNull(key, "key");
        int open = key.indexOf('{');
        if (open < 0) {
            return key;
        }
        int close = key.indexOf('}', open + 1);
        return close > open + 1 ? key.substring(open + 1, close) : key;
    }

    record Lease(String ownerToken, long expiresAtMillis, long fencingToken) {
        Lease {
            Objects.requireNonNull(ownerToken, "ownerToken");
        }

        boolean ownedBy(String token, long nowMillis) {
            return ownerToken.equals(token) && nowMillis < expiresAtMillis;
```

JAVA (continued 3/3)

```
        }
    }

    static int encodedUtf8Length(String value) {
        return value.getBytes(StandardCharsets.UTF_8).length;
    }

    public static void main(String[] args) {
        assert chooseStructure(new Requirement(true, false, false, false)) == Structure.SET;
        assert chooseStructure(new Requirement(false, true, false, false)) == Structure.SORTED_SET;
        assert chooseStructure(new Requirement(false, false, false, true)) == Structure.STREAM;

        long ttl = deterministicTtlSeconds("order:42", 300, 30);
        assert ttl >= 300 && ttl <= 330;

        CacheValue value = new CacheValue("paid", 4, 1_000);
        assert value.freshAt(999);
        assert !value.freshAt(1_000);

        assert clusterHashTag("order:{42}:items").equals("42");
        assert clusterHashTag("ordinary-key").equals("ordinary-key");

        Lease lease = new Lease("owner-a", 500, 7);
        assert lease.ownedBy("owner-a", 499);
        assert !lease.ownedBy("owner-b", 499);
        assert !lease.ownedBy("owner-a", 500);
        assert encodedUtf8Length("\u20B9") == 3;

        System.out.println("RedisInterviewCompanion checks passed");
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: structures and commands
- Interview Core: caching, TTL, and rate limits
- SDE-2 Follow-up: consistency, stampedes, and operations

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: FW 07 - MongoDB for Java Backend Interviews

Series index

Next book: FW 09 - Persistence, SQL, JPA, and Caching

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: https://github.com/vinayreddykalluri/SDE2-Interview-Handbook. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 08 of 12 - Study Step FW-08. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.